

Introduction

Goal: Realizzazione in Java del [Fireflysynch](#) limitatamente al requisito 2

Requirements

Costruire il simulatore di un sistema costituito da 3 lucciole. Inizialmente, le lucciole lampeggiamo ciascuna con una frequenza (costante) propria.

- dopo 10 secondi le lucciole devono lampeggiare in modo sincronizzato
- il sistema è dotato anche di un sonar . Quando il sonar rileva un ostacolo a una distanza minore di una distanza prefissata DMIN, le lucciole devono lampeggiare in modo sincronizzato. Quando la distanza dell’ostacolo (ri)supera DMIN, le lucciole tornano a lampeggiare in modo random

Requirement analysis

L'analisi estende i concetti di base includendo la reattività a stimoli ambientali variabili:

- Sonar:** Un dispositivo di input che produce dati sulla distanza di un ostacolo.
- DMIN:** Valore soglia (prefissato a 50 unità nel codice) che determina il cambio di stato del sistema.
- Modalità Random:** Comportamento in cui ogni lucciola opera con un `Timer` generato casualmente (tra 1000ms e 3000ms).
- Modalità Sincronizzata:** Comportamento coordinato attivato da un evento broadcast `startSync` che impone un periodo uniforme a tutte le istanze.

Problem analysis

La sfida principale risiede nella gestione della **transizione bidirezionale** tra comportamento autonomo e coordinato basata su eventi asincroni.

Architettura Logica

Il [sistema](#) viene modellato con tre tipi di attori nel contesto `ctxfirefly`:

- Sonar:** Attore proattivo che campiona la distanza. Se `Distanza < DMIN` emette l'evento `startSync`, altrimenti emette `stopSync`.
- Firefly:** Attore `dynamicOnly`. Gestisce la preemption del timer locale tramite la ricezione degli eventi del sonar, permettendo il passaggio immediato tra i due stati.
- Creator:** Responsabile della creazione dinamica delle 3 istanze di lucciola.

Formalizzazione dei Messaggi

```
Event startSync : startSync(PERIOD) // Forza la sincronizzazione su un periodo P
Event stopSync  : stopSync(0)       // Ripristina il comportamento random
Dispatch cellstate : cellstate(X,Y,COLOR) // Aggiornamento visuale per la Grid
```

Logica delle Transizioni (FSM Firefly)

La lucciola implementa una macchina a stati capace di reagire agli eventi del sonar durante l'attesa del prossimo flash:

```
Transition t0
  whenTimeVar Timer -> flash           // Ciclo di lampeggio corrente
  whenEvent startSync -> handleStartSync // Passaggio a modo sincronizzato
  whenEvent stopSync -> handleStopSync  // Ritorno a modo random
```

Test plans

Project

Testing

Deployment

Maintenance

